```
Output:
NCC-1701-D
bool(false)
NCC-1701-D
```

The code above demonstrates `strstr()` and `stristr()` in action, and it's straightforward. We search for the sub-string `"NCC"`, which exists in input string `$ship`, so the return is the remaining portion of `$ship`, including the search pattern. Using `strstr()` to search for lowercase `"ncc"` returns boolean `false`, but `stristr()` is quite happy to map lowercase ASCII letters to uppercase, as we can see in the final line. As always, remember that this is simply a byte-level comparison with some extra mapping rules for the upper-to-lowercase logic of `stristr()`. The following demonstrates binary and multi-byte safety:

```
$ship = "Registration: \x00NCC-1701-D";
print strstr($ship, "\x00NCC") . PHP_EOL;

$name = "Name: \u{141}ukasz";
print strstr($name, "\u{141}") . PHP_EOL;

var_dump(stristr($name, "\u{142}"));

Output:
NCC-1701-D
Łukasz
bool(false)
```

First, we can see that `strstr()` will handle null bytes without any issues, like the vast majority of PHP's core string functions. `stristr()` is similarly binary-safe, although that is not shown here. Secondly, we can see that although `strstr()` is not multi-byte aware, a simple binary comparison of two UTF-8 strings yields correct results. However, the third example would require `stristr()` to be encoding-aware in order to map U+141 (uppercase Polish Ł) to U+142 (lowercase Polish ł) in order to succeed. `stristr()` is only aware of the ASCII English alphabet and therefore returns boolean `false`. We'll cover the multi-byte safe forms of these functions in a moment.

Finally, if you wish to return the portion of the input string up to (rather than from) the first occurrence of the search string, simply add a boolean `true` as the third argument. `strstr($ship, 'NCC', true);` would therefore return the string